

MUSIC PLAYLIST AND LISTENING STATS MANAGER

ABSTRACT :

The **Music Playlist and Listening Stats Manager** is a Java-based console application designed to demonstrate fundamental programming concepts, control flow structures, modular design, recursion, and 1D-array manipulation. The system enables users to manage digital audio tracks, track listen counts and song durations, calculate aggregated statistics (such as total listening time, average track duration, and top-played tracks), and filter songs based on playback counts. By integrating core syllabus topics—such as primitive data types, expressions, formatted console I/O, control flow statements, overloaded methods, and recursive analysis—this project models a practical digital media management platform using standard Java programming paradigms.

ADVANTAGES AND DISADVANTAGES :

Advantages

- **Memory Efficiency:** Utilizes lightweight 1D arrays and primitive data types, minimizing dynamic memory overhead.
- **Modular Architecture:** Uses static helper methods for task decomposition, facilitating clean code maintenance and debuggability.
- **Interactive CLI Interface:** Leverages standard console I/O via Scanner and formatted printf strings for intuitive menu-driven interaction.
- **Comprehensive Analytics:** Provides algorithmic tracking for total duration, average play times, min/max statistics, and play-count thresholds.

Disadvantages

- **Fixed Data Capacity:** Relies on fixed-size 1D arrays without standard library dynamic collections like `ArrayList`, restricting playlist expansion beyond initial capacity limits.
- **In-Memory Storage:** Data is retained strictly during runtime and resets upon program termination due to lack of file or database persistence.
- **Single-Threaded Execution:** Processing occurs sequentially, which may limit scalability for massive dataset evaluations.

```
import java.util.Scanner;
```

```
/**
```

- Project: Music Playlist & Listening Stats Manager
- Demonstrating Modules 1, 2, and 3 concepts.

```
*/ public class PlaylistManager {
```

```
// Global Constants (Module 1)
```

```
private static final int MAX_PLAYLIST_SIZE = 5;
```

```
private static final double SECONDS_TO_MINUTES_RATIO = 60.0;
```

```
public static void main(String[] args) {
```

```
    // Module 1: Java Environment, Variables, Primitive Data Types  
    Scanner scanner = new Scanner(System.in);
```

```
    // Parallel 1D-Arrays for Data Storage (Module 3)
```

```
    int[] trackIds = new int[MAX_PLAYLIST_SIZE];
```

```
    int[] playCounts = new int[MAX_PLAYLIST_SIZE];
```

```
    double[] durationsInSeconds = new double[MAX_PLAYLIST_SIZE];
```

```
    // Default Initialization Data
```

```
    int trackCount = 0;
```

```
    // Populate initial dataset manually (Module 1 Literals &  
Assignment)
```

```
    trackIds[0] = 101; playCounts[0] = 45; durationsInSeconds[0] =  
215.5; trackCount++;
```

```
    trackIds[1] = 102; playCounts[1] = 12; durationsInSeconds[1] =
```

```

180.0; trackCount++;
    trackIds[2] = 103; playCounts[2] = 88; durationsInSeconds[2] =
240.2; trackCount++;
    trackIds[3] = 104; playCounts[3] = 0; durationsInSeconds[3] =
195.8; trackCount++;
    trackIds[4] = 105; playCounts[4] = 30; durationsInSeconds[4] =
310.0; trackCount++;

    boolean keepRunning = true; // Module 1 Variable

    // Module 2: Control Flow - Selection & Iteration (while loop)
    while (keepRunning) {
        displayMenu();
        System.out.print("Enter choice (1-8): ");

        if (!scanner.hasNextInt()) {
            System.out.println("Invalid input! Please enter a
number.");
            scanner.next(); // Clear invalid token
            continue; // Module 2: continue keyword
        }

        int choice = scanner.nextInt();

        // Module 2: Switch-default, break
        switch (choice) {
            case 1:
                displayPlaylist(trackIds, playCounts,
durationsInSeconds, trackCount);
                break;
            case 2:
                // Module 1 & 3: Console Input & Search
                System.out.print("Enter Track ID to update plays: ");
                int idToUpdate = scanner.nextInt();
                int index = searchTrackIndex(trackIds, trackCount,
idToUpdate);

                // Module 2: Selection Statement (if-else)
                if (index != -1) {

```

```

        System.out.print("Enter additional plays to add:
");

        int addedPlays = scanner.nextInt();
        // Module 1: Assignment and Unary/Arithmetic
Operator
        playCounts[index] += addedPlays;
        System.out.printf("Successfully updated Track
ID %d. New Play Count: %d\n", idToUpdate, playCounts[index]);
    } else {
        System.out.println("Track ID not found!");
    }
    break;
case 3:
    // Module 3: Summation and Formatted Printing
    double totalDurationSec =
calculateTotalDuration(durationsInSeconds, trackCount);
    double totalDurationMin = totalDurationSec /
SECONDS_TO_MINUTES_RATIO; // Module 1 Expression Evaluation
    System.out.printf("\nTotal Duration: %.2f seconds
(%.2f minutes)\n", totalDurationSec, totalDurationMin);
    break;
case 4:
    // Module 3: Average calculation
    double avgDuration =
calculateAverageDuration(durationsInSeconds, trackCount);
    System.out.printf("\nAverage Track Duration: %.2f
seconds\n", avgDuration);
    break;
case 5:
    // Module 3: Counting Technique with Overloaded Method
    System.out.print("Enter minimum play count threshold:
");

    int threshold = scanner.nextInt();
    int popularCount =
countTracksAboveThreshold(playCounts, trackCount, threshold);
    System.out.printf("Tracks with plays > %d: %d\n",
threshold, popularCount);
    break;
case 6:

```

```

        // Module 3: Method Overloading (Default threshold =
20)
        int defaultPopularCount =
countTracksAboveThreshold(playCounts, trackCount);
        System.out.printf("Tracks exceeding default popularity
(20 plays): %d\n", defaultPopularCount);
        break;
    case 7:
        // Module 3: Recursion Analysis
        System.out.print("Enter play count threshold to count
recursively: ");
        int recThreshold = scanner.nextInt();
        int recCount = countTracksRecursive(playCounts,
trackCount, recThreshold, 0);
        System.out.printf("[Recursive Result] Tracks
exceeding %d plays: %d\n", recThreshold, recCount);
        break;
    case 8:
        System.out.println("Exiting Playlist Manager.
Goodbye!");
        keepRunning = false;
        break;
    default:
        System.out.println("Invalid option! Please enter a
value between 1 and 8.");
    }
    System.out.println();
}

scanner.close();
}

// =====
// MODULE 3: MODULAR PROGRAMMING & METHODS
// =====

/**
 * Displays main menu options using formatted console output.
 */

```

```

private static void displayMenu() {
    System.out.println("===== MUSIC PLAYLIST & STATS MANAGER
=====");
    System.out.println("1. View Playlist Tracks");
    System.out.println("2. Update Track Play Count");
    System.out.println("3. Calculate Total Playlist Duration");
    System.out.println("4. Calculate Average Track Duration");
    System.out.println("5. Count Popular Tracks (Custom Threshold)");
    System.out.println("6. Count Popular Tracks (Default Threshold =
20)");
    System.out.println("7. Recursive Track Play Count Analysis");
    System.out.println("8. Exit");

    System.out.println("=====
==");
}

/**
 * Module 3: 1D Array Traversal & Enhanced For-Each Loop Demonstration
 */
private static void displayPlaylist(int[] ids, int[] plays, double[]
durations, int count) {
    System.out.println("\n--- CURRENT PLAYLIST ---");
    // Module 1: Formatted Printing
    System.out.printf("%-10s %-12s %-15s %-10s\n", "Track ID", "Play
Count", "Duration (s)", "Status");
    System.out.println("-----
----");

    // Module 2 & 3: Standard For Loop Traversal
    for (int i = 0; i < count; i++) {
        // Module 1: Ternary Operator
        String status = (plays[i] > 50) ? "Trending" : "Regular";
        System.out.printf("%-10d %-12d %-15.1f %-10s\n", ids[i],
plays[i], durations[i], status);
    }

    // Demonstration of Module 3: Enhanced For-Each Loop for simple
print output

```

```

        System.out.print("\nRegistered Track IDs: ");
        for (int id : ids) {
            System.out.print(id + " ");
        }
        System.out.println();
    }

/**
 * Module 3: Linear Search Algorithm on 1D Array
 */
private static int searchTrackIndex(int[] ids, int count, int
targetId) {
    for (int i = 0; i < count; i++) {
        if (ids[i] == targetId) { // Module 1: Relational Operator
            return i; // Found
        }
    }
    return -1; // Not Found
}

/**
 * Module 3: Summation Technique over Array Elements
 */
private static double calculateTotalDuration(double[] durations, int
count) {
    double total = 0.0;
    int i = 0;
    // Module 2: do-while loop
    if (count > 0) {
        do {
            total += durations[i];
            i++;
        } while (i < count);
    }
    return total;
}

/**
 * Module 3: Calculating Average from Summation

```

```

    */
private static double calculateAverageDuration(double[] durations, int
count) {
    if (count == 0) return 0.0;
    double total = calculateTotalDuration(durations, count); // Method
Invocation
    // Module 1: Typecasting & Arithmetic
    return total / (double) count;
}

/**
 * Module 3: Counting Technique based on Condition
 */
private static int countTracksAboveThreshold(int[] plays, int count,
int minThreshold) {
    int matchCount = 0;
    for (int i = 0; i < count; i++) {
        if (plays[i] > minThreshold) {
            matchCount++; // Unary Operator
        }
    }
    return matchCount;
}

/**
 * Module 3: Method Overloading (Same method name, different parameter
signature)
 */
private static int countTracksAboveThreshold(int[] plays, int count) {
    int defaultThreshold = 20; // Default constant
    return countTracksAboveThreshold(plays, count, defaultThreshold);
}

/**
 * Module 3: Recursion Implementation (Base Case & Recursive Case)
 */
private static int countTracksRecursive(int[] plays, int count, int
threshold, int index) {
    // Base Case 1: End of array reached

```

```

    if (index >= count) {
        return 0;
    }

    // Recursive Step Calculation
    int currentMatch = (plays[index] > threshold) ? 1 : 0;

    // Recursive Case: Function invokes itself with incremented index
    return currentMatch + countTracksRecursive(plays, count,
threshold, index + 1);
}

}

```

OUTPUT

```

===== MUSIC PLAYLIST & STATS MANAGER =====
1. View Playlist Tracks
2. Update Track Play Count
3. Calculate Total Playlist Duration
4. Calculate Average Track Duration
5. Count Popular Tracks (Custom Threshold)
6. Count Popular Tracks (Default Threshold = 20)
7. Recursive Track Play Count Analysis
8. Exit
=====
Enter choice (1-8): 1

--- CURRENT PLAYLIST ---
Track ID   Play Count   Duration (s)   Status
-----
101         45           215.5         Regular
102         12           180.0         Regular
103         88           240.2         Trending
104          0           195.8         Regular
105         30           310.0         Regular

Registered Track IDs: 101 102 103 104 105

```

```
===== MUSIC PLAYLIST & STATS MANAGER =====
...
Enter choice (1-8): 2
Enter Track ID to update plays: 101
Enter additional plays to add: 10
Successfully updated Track ID 101. New Play Count: 55

===== MUSIC PLAYLIST & STATS MANAGER =====
...
Enter choice (1-8): 3

Total Duration: 1141.50 seconds (19.03 minutes)

===== MUSIC PLAYLIST & STATS MANAGER =====
...
Enter choice (1-8): 7
Enter play count threshold to count recursively: 30
[Recursive Result] Tracks exceeding 30 plays: 3

===== MUSIC PLAYLIST & STATS MANAGER =====
...
Enter choice (1-8): 8
Exiting Playlist Manager. Goodbye!
```

CONCLUSION & FUTURE WORK :

Conclusion

The **Music Playlist and Listening Stats Manager** successfully models the essential features of a music playlist system while demonstrating fundamental computational logic and Java programming concepts. The implementation exercises key programmatic controls, including structured primitive type operations, operator evaluations, Scanner-driven input processing, formatted printf tables, switch-case constructs, array traversals, overloaded static methods, and recursive evaluation.

Future Work

- **Dynamic Memory Expansion:** Replace static 1D arrays with standard dynamic collections (`ArrayList`) to support unlimited song additions at runtime.
- **File I/O Persistence:** Implement file reading/writing operations (`BufferedReader` / `BufferedWriter`) to persist playlist metadata across execution sessions.
- **Object-Oriented Refactoring:** Enapsulate song attributes (`id`, `title`, `duration`, `playCount`) into dedicated `Track` classes utilizing object-oriented principles.
- **Graphical User Interface (GUI):** Upgrade the current command-line interface to a visual UI framework such as `JavaFX` or `Swing`.